

AnyCloud平台AIC8800DL USB WIFI模组适配指导手册

前言

AIC8800DL是AIC出的一款USB接口的WIFI芯片，支持2.4GHz Wi-Fi6，支持蓝牙功能。

现基于AnyCloud平台对AIC8800DL在AV100 PDK1.07上的适配进行简要说明。

注：本适配文档使用的驱动源码包为：aic8800d_linux_sdk_V3.0_2023_1219_3cf85031.7z。如果用户使用的驱动源码包

不一致，适配过程可能和此文档的描述有一些出入。

WIFI驱动编译

使用原厂提供wifi源码，修改Makefile文件，即可编译出相关驱动。

解压驱动包

将压缩包拷贝到以下目录（路径为以SDK根目录为参照的相对路径）：

os/driver_src/wifi/

进入此目录，解压压缩包。

```
cd os/driver_src/wifi/  
7z x aic8800d_linux_sdk_V3.0_2023_1219_3cf85031.7z -r  
cd aic8800_porting_package/USB
```

厂商提供的驱动包中，默认编译的目标平台为ubuntu，因此需要修改驱动包中的Makefile文件。

打开aic8800_porting_package/USB/driver_fw/drivers/aic8800/Makefile，按照如下修改：

将CONFIG_PLATFORM_UBUNTU设置为n。

```
# Platform support list  
CONFIG_PLATFORM_ROCKCHIP ?= n  
CONFIG_PLATFORM_ALLWINNER ?= n  
CONFIG_PLATFORM_AMLOGIC ?= n  
CONFIG_PLATFORM_UBUNTU ?= n
```

同样地，也要确认和修改蓝牙部分的Makefile：

打开aic8800_porting_package/USB/driver_fw/drivers/aic_btusb/Makefile，按照如下修改：

将CONFIG_PLATFORM_UBUNTU设置为 n。

```
# Platform support list
CONFIG_PLATFORM_ROCKCHIP ?= n
CONFIG_PLATFORM_ALLWINNER ?= n
CONFIG_PLATFORM_AMLOGIC ?= n
CONFIG_PLATFORM_UBUNTU ?= n
```

打开aic8800_porting_package/USB/driver_fw/drivers/aic8800/aic8800_fdrv/Makefile，按照如下修改：

```
CONFIG_VRF_DCDC_MODE = y
#修改为：
CONFIG_VRF_DCDC_MODE = n

#如果需要使用到iwconfig/iwlist工具：
CONFIG_USE_WIRELESS_EXT = n
#修改为：
CONFIG_USE_WIRELESS_EXT = y
```

修改驱动源码

修改bt驱动源码：

由于我们使用bluez蓝牙协议栈作为蓝牙驱动的上层应用测试，而aic的bt驱动支持bluez和bluedroid两种，因此需要修改bt驱动源码，以支持bluez。

打开aic8800_porting_package/USB/driver_fw/drivers/aic_btusb/aic_btusb.h，按照如下修改：

修改CONFIG_BLUEDROID宏定义为0。

```
#ifndef CONFIG_PLATFORM_UBUNTU
#define CONFIG_BLUEDROID    1 /* bleuz 0, bluedroid 1 */
#else
#define CONFIG_BLUEDROID    0 /* bleuz 0, bluedroid 1 */
#endif
```

如果需要打开bt驱动更多调试信息，可以在aic_btusb.h中，设置AICBT_DBG_FLAG为1。

```
#define AICBT_DBG_FLAG      1
```

编译前，需要将aic_btusb.c和aic_btusb.h放到kernel/drivers/bluetooth/下。
接下来开始编译。

编写编译脚本

查阅《AIC8800_USB_porting_guide_v1_1_20220526.pdf》手册，需要同时编译aic_load_fw.ko和aic8800_fdrv.ko，避免过度修改Makefile文件，使用编译脚本来编译两个驱动。

配置相关宏

根据指导手册需要打开两个宏CONFIG_AIC_LOADFW_SUPPORT和CONFIG_AIC8800_WLAN_SUPPORT，表明编译出aic_load_fw.ko和aic8800_fdrv.ko。

```
export CONFIG_AIC_LOADFW_SUPPORT=m
export CONFIG_AIC8800_WLAN_SUPPORT=m

export CONFIG_BT_AICBTUSB=m
```

指定内核编译路径、芯片架构及交叉编译器

```
export KDIR=/home/$(whoami)/anyka_sdk/anycld/AnyCloud39AV100_SDK_V1.07/os/kbd
export ARCH=arm
export CROSS_COMPILE=arm-anycld-linux-uclibcgnueabi-
```

make命令编译传参

```
# aic_load_fw.ko and aic8800_fdrv.ko
make -C $KDIR M=$PWD/drivers/aic8800/ ARCH=arm CROSS_COMPILE=${CROSS_COMPILE}
# aic_btusb.ko
make -C $KDIR M=$PWD/drivers/aic_btusb/ ARCH=arm CROSS_COMPILE=${CROSS_COMPILE}
```

完整的shell脚本

```
#!/bin/sh

# WIFI
export CONFIG_AIC_LOADFW_SUPPORT=m
export CONFIG_AIC8800_WLAN_SUPPORT=m

# BT
export CONFIG_BT_AICBTUSB=m

# AV100
export KDIR=/home/$(whoami)/anyka_sdk/anycloud/AnyCloud39AV100_SDK_V1.07/os/kbd
export ARCH=arm
export CROSS_COMPILE=arm-anycloud-linux-uclibcgnueabi-

# aic_load_fw.ko and aic8800_fdv.ko
make -C $KDIR M=$PWD/drivers/aic8800/ ARCH=arm CROSS_COMPILE=${CROSS_COMPILE}

# aic_btusb.ko
make -C $KDIR M=$PWD/drivers/aic_btusb/ ARCH=arm CROSS_COMPILE=${CROSS_COMPILE}

# install
mkdir -p out/av100/
rm -rf out/av100/*
find drivers -name '*.ko' -exec cp {} out/av100/ \;
```

使用脚本编译驱动

将shell脚本保存为build_aic8800.sh，赋予可执行权限，“chmod +x build_aic8800.sh”，执行后，即可编译出aic_load_fw.ko、aic8800_fdrv.ko和aic_btusb.ko。

固件文件

加载驱动依赖于固件文件，将aic8800_porting_package/USB/driver_fw/fw/aic8800DC文件，拷贝到设备/lib/firmware目录。

正常只要如下几个文件即可：

```
ls -l /lib/firmware/aic8800DC/  
aic_userconfig_8800dc.txt  
fmacfw_patch_8800dc.bin  
fmacfw_patch_8800dc_ipc.bin  
fmacfw_patch_8800dc_ipc_u02.bin  
fmacfw_patch_8800dc_u02.bin
```

驱动加载

将编译好的aic_load_fw.ko和aic8800_fdrv.ko驱动文件拷贝到设备根文件系统，按照如下命令，依次加载驱动。

```
modprobe cfg80211  
modprobe usb-common  
modprobe usbcore  
insmod /usr/modules/ak_hcd.ko  
insmod /usr/modules/firmware_class.ko  
insmod /usr/modules/aic_load_fw.ko aic_fw_path=/lib/firmware  
insmod /usr/modules/aic8800_fdrv.ko
```

配网功能

查看wlan0节点

加载完毕后，使用ifconfig命令会有wlan0节点

```
wlan0  Link encap:Ethernet HWaddr 38:73:EA:E7:DA:00  
        BROADCAST MULTICAST MTU:1500 Metric:1  
        RX packets:0 errors:0 dropped:0 overruns:0 frame:0  
        TX packets:0 errors:0 dropped:0 overruns:0 carrier:0  
        collisions:0 txqueuelen:1000  
        RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

配网

执行如下命令后，ping路由器网关正常则表明配网无异常。

```
wpa_supplicant -Dnl80211 -iwlan0 -c /etc/config/wpa_supplicant.conf -B  
route del default  
udhcpc -t 30 -T 1 -i wlan0 -b
```

注意：/etc/config/wpa_supplicant.conf文件需填写无线路由器的ssid和密码。

扫描热点

在配网前，进行热点扫描，如果可以扫描到周围wifi热点，则表明硬件和驱动正常。

常规命令

```
iwlist wlan0 scanning
```

使用wpa_cli名，使用wpa_cli则需要运行了wpa_supplicant服务后才能执行。

```
wpa_cli -i wlan0 scan  
wpa_cli -i wlan0 scan_result
```

吞吐量测试

iperf3打流

办公室环境打流，2.4G上行40.0Mbits/sec，下行34Mbits/sec；

开发板当客户端：

```
[root@anyka /tmp/nfs/av100/pdk1.04/aic8800]$ iperf3-static -c 192.168.10.187 -i 1 -t 10
```

Connecting to host 192.168.10.187, port 5201

[5] local 192.168.10.31 port 42504 connected to 192.168.10.187 port 5201

[ID]	Interval	Transfer	Bitrate	Retr	Cwnd
[5]	0.00-1.00 sec	5.03 MBytes	42.2 Mbites/sec	0	171 KBytes
[5]	1.00-2.00 sec	5.03 MBytes	42.3 Mbites/sec	0	171 KBytes
[5]	2.00-3.00 sec	5.15 MBytes	43.2 Mbites/sec	0	171 KBytes
[5]	3.00-4.00 sec	4.98 MBytes	41.6 Mbites/sec	1	171 KBytes
[5]	4.00-5.00 sec	4.72 MBytes	39.7 Mbites/sec	2	171 KBytes
[5]	5.00-6.01 sec	4.64 MBytes	38.7 Mbites/sec	2	171 KBytes
[5]	6.01-7.00 sec	5.25 MBytes	44.3 Mbites/sec	0	171 KBytes
[5]	7.00-8.00 sec	4.70 MBytes	39.4 Mbites/sec	0	171 KBytes
[5]	8.00-9.01 sec	4.26 MBytes	35.4 Mbites/sec	1	171 KBytes
[5]	9.01-10.00 sec	4.46 MBytes	37.7 Mbites/sec	1	171 KBytes

[ID]	Interval	Transfer	Bitrate	Retr	
[5]	0.00-10.00 sec	48.2 MBytes	40.5 Mbites/sec	7	sender
[5]	0.00-10.00 sec	48.1 MBytes	40.3 Mbites/sec		receiver

iperf Done.

开发板当服务端:

```

iperf3 -s
-----
Server listening on 5201
-----
Accepted connection from 192.168.10.187, port 1039
[ 5] local 192.168.10.31 port 5201 connected to 192.168.10.187 port 1040
[ ID] Interval      Transfer   Bitrate
[ 5]  0.00-1.00  sec  3.16 MBytes  26.5 Mbits/sec
[ 5]  1.00-2.00  sec  4.05 MBytes  34.0 Mbits/sec
[ 5]  2.00-3.00  sec  3.72 MBytes  31.2 Mbits/sec
[ 5]  3.00-4.00  sec  4.16 MBytes  34.9 Mbits/sec
[ 5]  4.00-5.00  sec  4.36 MBytes  36.5 Mbits/sec
[ 5]  5.00-6.00  sec  4.63 MBytes  38.8 Mbits/sec
[ 5]  6.00-7.00  sec  3.82 MBytes  32.0 Mbits/sec
[ 5]  7.00-8.00  sec  3.75 MBytes  31.5 Mbits/sec
[ 5]  8.00-9.00  sec  4.38 MBytes  36.7 Mbits/sec
[ 5]  9.00-10.00 sec  4.60 MBytes  38.5 Mbits/sec
[ 5] 10.00-10.02 sec  93.8 KBytes  52.3 Mbits/sec
-----
[ ID] Interval      Transfer   Bitrate
[ 5]  0.00-10.02 sec  40.7 MBytes  34.1 Mbits/sec          receiver

```

蓝牙使用

打开蓝牙子系统

进入到内核目录，可以通过make O=../bd menuconfig方式进行配置

```

[*] Networking support --->
<M> Bluetooth subsystem support --->
[*] Bluetooth Classic (BR/EDR) features
[*] Bluetooth Low Energy (LE) features
Bluetooth device drivers --->

```

如果需要调试蓝牙，可以加入蓝牙的debugfs。

```

[*] Export Bluetooth internals in debugfs

```

配置完成后，执行make O=../bd modules命令后，内核会生成crc16.ko、bluetooth.ko。它们的路径如下：


```
../bd/lib/crc16.ko
../bd/net/bluetooth/bluetooth.ko
```

蓝牙驱动加载

将编译好的驱动文件和上一步生成的蓝牙子系统ko文件都拷贝到设备根文件系统/usr/modules/，依次加载

```
insmod /usr/modules/crc16.ko
insmod /usr/modules/bluetooth.ko
insmod /usr/modules/aic_btusb.ko
```

查看蓝牙hci0节点

使用bluez工具包前需要交叉编译bluez。本文不讲述bluez的编译过程，请参考相关文档编译。
将编译好的bluez工具包，放置到T卡或者其他目录，设定环境变量，让系统识别到hciconfig等工具。
环境变量设置参考脚本，脚本名假设：export.sh。

```
#!/bin/sh

CUR_DIR=$(pwd)

BLUEZ_PATH=${CUR_DIR}/_install/usr/local

export LD_LIBRARY_PATH=${BLUEZ_PATH}/lib:$LD_LIBRARY_PATH
export PATH=${BLUEZ_PATH}/libexec/bluetooth:$PATH
export PATH=${BLUEZ_PATH}/bin:$PATH
```

注意环境变量脚本路径，使用点命令让其生效。

```
../export.sh
```

查看节点

蓝牙驱动加载完毕后，即可使用hciconfig工具查看节点。

```
[root@anyka ]$ hciconfig -a
hci0:  Type: BR/EDR  Bus: USB
       BD Address: 38:73:EA:E6:58:1B  ACL MTU: 1021:9  SCO MTU: 255:4
       DOWN
       RX bytes:616 acl:0 sco:0 events:34 errors:0
       TX bytes:375 acl:0 sco:0 commands:34 errors:0
       Features: 0xbf 0x2e 0x4d 0xfe 0xd8 0x3f 0x7b 0x87
       Packet type: DM1 DM3 DM5 DH1 DH3 DH5 HV1 HV3
       Link policy: RSWITCH SNIFF
       Link mode: SLAVE ACCEPT
```

蓝牙连接

为了使bluez工具正常使用，需要将部分配置文件拷贝到对应目录。

- 1.将附件bluez目录的etc/dbus-1目录拷贝到设备的/etc目录，由于anyka平台/etc目录是不可写的，需要在制作rootfs时加入；
- 2.将附件bluez目录的usr/local/share/dbus-1拷贝到/usr/local/share目录，同样在nor flash要在制作rootfs时加入，nand flash可以读写则直接拷贝；
- 3.新增dbus用户，用于dbus运行，默认是messagebus，编译时设定为dbus了。

```
echo dbus:x:500:500::/home/dbus:/bin/sh >> /etc/passwd
```

- 4.系统启动时，在/var目录创建run/dbus目录

```
mkdir -p /var/run/dbus
```

- 5.运行dbus和bluetoothd服务

运行前必须要执行“hciconfig hci0 up”，让蓝牙节点起来。

```
hciconfig hci0 up
dbus-daemon --system --print-pid --print-address
bluetoothd -n -d &
```

6.执行bluetoothctl，进入到bluetoothctl的命令窗口中。

```
bluetoothctl
```

在命令窗口中，输入help，查看所支持的命令：

```
[bluetooth]# help
Available commands:
list                List available controllers
show [ctrl]         Controller information
select <ctrl>       Select default controller
devices             List available devices
paired-devices      List paired devices
power <on/off>      Set controller power
pairable <on/off>   Set controller pairable mode
discoverable <on/off> Set controller discoverable mode
agent <on/off/capability> Enable/disable agent with given capability
default-agent       Set agent as the default one
scan <on/off>       Scan for devices
info <dev>          Device information
pair <dev>          Pair with device
trust <dev>         Trust device
untrust <dev>       Untrust device
block <dev>         Block device
unblock <dev>       Unblock device
remove <dev>        Remove device
connect <dev>       Connect device
disconnect <dev>    Disconnect device
version            Display version
quit              Quit program
```

7.在bluetoothctl窗口中使用discoverable on可以打开可被发现和可被连接功能，但是不能发起广播。要开启广播，可以使用hciconfig命令。

打开广播：

```
hciconfig hci0 leadv 0
```

关闭广播：

```
hciconfig hci0 noleadv
```

测试蓝牙通信

使用手机蓝牙测试工具：nRF Connect，测试蓝牙的通信功能。

手机下载nRF Connect，然后打开测试工具，在SCANNER选项中搜索名为：Bluez 5.18的蓝牙设备（设备需要先启动广播），点击CONNECT进行连接。

连接成功后设备打印：

```
bluetoothd[459]: src/adapter.c:connected_callback() hci0 device 53:CD:3C:4F:4E:B6 connected eir_len 0
bluetoothd[459]: src/device.c:device_create() dst 53:CD:3C:4F:4E:B6
bluetoothd[459]: src/device.c:device_new() address 53:CD:3C:4F:4E:B6
bluetoothd[459]: src/device.c:device_new() Creating device /org/bluez/hci0/dev_53_CD_3C_4F_4E_B6
bluetoothd[459]: src/device.c:btd_device_set_temporary() temporary 1
bluetoothd[459]: src/adapter.c:adapter_connect_list_remove() device /org/bluez/hci0/dev_53_CD_3C_4F_4E_B6 is not on
the list, ignoring
bluetoothd[459]: src/attrib-server.c:connect_event()
bluetoothd[459]: attrib/gattrib.c:g_attrib_ref() 0x190dce8: ref=1
bluetoothd[459]: Refusing storage path for private addressed device /org/bluez/hci0/dev_53_CD_3C_4F_4E_B6
bluetoothd[459]: attrib/gattrib.c:g_attrib_ref() 0x190dce8: ref=2
```

在nRF Connect中，选择CLIENT选项，在这里可以查看到设备公开的各种属性。我们可以通过读写这些属性来验证蓝牙的通信功能是否正常。

结束语

以上即是AIC8800DL模组在AnyCloud平台的适配。